Enhanced LLM-Aided Testbench Generation

1. Examples

Example 1: 2 to 1 multiplexer

Example 2: 4-it Adder

Own module: priority 4-bit encoder

2. Summarize

- **Run 1: 2-to-1 Multiplexer (mux2to1)**

DUT interface

Module name: mux2to1

Inputs:

a : 1 bit

b : 1 bit

sel : 1 bit

Output:

y : 1 bit

Functional behavior

This DUT is a 2-to-1 combinational multiplexer. When sel = 0, the output y should equal a. When sel = 1, the output y should equal b.

Verilog DUT

```
module mux2to1 (
    input wire a,
    input wire b,
    input wire sel,
    output wire y
);
    assign y = sel ? b : a;
endmodule
```

Icarus Verilog commands

```
iverilog -o sim_mux.vvp mux2to1.v testbench_final.v
```

vvp sim_mux.vvp

- Run 2: 4-bit Adder (adder4bit)

DUT interface

Module name: adder4bit

Inputs:

a : 4 bits

b : 4 bits

Outputs:

sum : 4 bits

carry : 1 bit

Functional behavior

This DUT performs unsigned 4-bit addition. The lower 4 bits of the result are assigned to sum, and the carry-out bit is assigned to carry. If a + b > 15, then carry should be 1.

Verilog DUT

```
module adder4bit (
    input wire [3:0] a,
    input wire [3:0] b,
    output wire [3:0] sum,
    output wire carry
);
    wire [4:0] result;
    assign result = a + b;
    assign sum = result[3:0];
    assign carry = result[4];
endmodule
```

Icarus Verilog commands

```
iverilog -o sim_adder.vvp adder4bit.v testbench_final.v
vvp sim_adder.vvp
```

- Run 3: 4-bit Priority Encoder (encoder4)

DUT interface

Module name: encoder4

Input:

in : 4 bits

Outputs:

pos : 2 bits

valid : 1 bit

Functional behavior

This DUT is a 4-bit priority encoder. Bit 3 has the highest priority, followed by bit 2, bit 1, and bit 0. If multiple bits are asserted, only the highest-priority asserted bit is encoded. If all input bits are 0, then valid = 0 and pos = 2'b00.

Verilog DUT

```
module encoder4 (
    input   [3:0] in,
    output reg [1:0] pos,
    output reg valid
);

always @(*) begin
    valid = 1'b1;
    casex (in)
        4'b1xxx: pos = 2'b11;
        4'b01xx: pos = 2'b10;
        4'b001x: pos = 2'b01;
        4'b0001: pos = 2'b00;
        default: begin
            pos = 2'b00;
            valid = 1'b0;
        end
    endcase
end
```

```
endmodule
```

Icarus Verilog commands

```
iverilog -o sim_encoder.vvp encoder4.v testbench_final.v
vvp sim_encoder.vvp
```

3. Explanation for Part II

- **Golden model (golden_model.py)**

The golden model for this example is:

```
def mux2to1_golden(a, b, sel):
    y = a if sel == 0 else b
    return {'y': y}
```

This golden model expects three input signals: a, b, and sel. It produces one output, y, as the reference output. The computation follows the logic of a 2-to-1 multiplexer: when sel = 0, the output y is assigned from a; when sel = 1, the output y is assigned from b. The result is returned as a dictionary in the form {'y': y}, which makes it easy to store the expected output for each test pattern.

- **Pattern file (test_patterns_with_golden.json)**

The pattern file contains a list of test cases. Each test case stores the DUT input values and the corresponding expected output generated by the golden model. For this example, each entry contains the fields a, b, and sel, as well as an expected_outputs field. The expected output is stored inside expected_outputs, for example:

```
{
  "a": 0,
  "b": 1,
  "sel": 1,
  "expected_outputs": {
    "y": 1
  }
}
```

This means that for the input combination a=0, b=1, and sel=1, the golden reference output is y=1. In this way, the JSON file stores both the input patterns and the expected outputs needed for automatic checking.

- **Updater effect: difference between testbench_initial.v and testbench_final.v**

One concrete difference between testbench_initial.v and testbench_final.v is that the final testbench

adds automatic checking logic. In testbench_initial.v, each test pattern only applies inputs and prints the observed DUT output using $display, but it does not determine whether the result is correct. In contrast, testbench_final.v compares the DUT output y against the expected value for each test case using conditions such as if (y === 0) or if (y === 1). It also prints pass/fail messages and keeps counters of passed_tests and failed_tests, followed by a final test summary. Therefore, the updater transforms the initial testbench from a simple stimulus-and-print testbench into a self-checking verification testbench.

4. Part IV

Here, 4'b1xxx: pos = 2'b11 has been changed to 2'b10

```
# Verilog module to be tested
encoder4_verilog_code = """
module encoder4 (
        input [3:0] in,
        output reg [1:0] pos,
        output reg valid
);

always @(*) begin
        valid = 1'b1;
        casex (in)
                4'b1xxx: pos = 2'b10;
                4'b01xx: pos = 2'b10;
                4'b001x: pos = 2'b01;
                4'b0001: pos = 2'b00;
                default: begin
                        pos = 2'b00;
                        valid = 1'b0;
                end
        endcase
end

endmodule

"""

print("Verilog Module:")
print("=" * 80)
print(encoder4_verilog_code)
print("=" * 80)

# Save the Verilog code for later simulation
with open(f"{output_dir}/encoder4.v", "w") as f:
        f.write(encoder4_verilog_code)
```

Only 13 tests passed, 7 failed

```
Running simulation...
Simulation Output:
================================================================================
Test 1: in = 0000
✓ Valid: 0, Pos: 00
Test 2: in = 0001
✓ Valid: 1, Pos: 00
Test 3: in = 0010
✓ Valid: 1, Pos: 01
Test 4: in = 0100
✓ Valid: 1, Pos: 10
Test 5: in = 1000
✗ Valid: 1, Pos: 10, Expected Valid: 1, Pos: 11
Test 6: in = 0011
✓ Valid: 1, Pos: 01
Test 7: in = 0110
✓ Valid: 1, Pos: 10
Test 8: in = 1100
✗ Valid: 1, Pos: 10, Expected Valid: 1, Pos: 11
Test 9: in = 0101
✓ Valid: 1, Pos: 10
Test 10: in = 1111
✗ Valid: 1, Pos: 10, Expected Valid: 1, Pos: 11
Test 11: in = 1010
✗ Valid: 1, Pos: 10, Expected Valid: 1, Pos: 11
Test 12: in = 1101
✗ Valid: 1, Pos: 10, Expected Valid: 1, Pos: 11
Test 13: in = 0111
✓ Valid: 1, Pos: 10
Test 14: in = 1011
✗ Valid: 1, Pos: 10, Expected Valid: 1, Pos: 11
Test 15: in = 1001
✗ Valid: 1, Pos: 10, Expected Valid: 1, Pos: 11
Test 16: in = 0110
✓ Valid: 1, Pos: 10
Test 17: in = 0011
✓ Valid: 1, Pos: 01
Test 18: in = 0100
✓ Valid: 1, Pos: 10
Test 19: in = 0010
✓ Valid: 1, Pos: 01
Test 20: in = 0001
✓ Valid: 1, Pos: 00
Test Summary: Total Tests = 20, Passed = 13, Failed = 7
```

After fixing, all test case passed

```
Compiling testbench...
✓ Compilation successful

Running simulation...
Simulation Output:
================================================================================
Test 1: in = 0000, pos = 00, valid = 0
✓ Test Passed: pos = 00, valid = 0
Test 2: in = 0001, pos = 00, valid = 1
✓ Test Passed: pos = 00, valid = 1
Test 3: in = 0010, pos = 01, valid = 1
✓ Test Passed: pos = 01, valid = 1
Test 4: in = 0100, pos = 10, valid = 1
✓ Test Passed: pos = 10, valid = 1
Test 5: in = 1000, pos = 11, valid = 1
✓ Test Passed: pos = 11, valid = 1
Test 6: in = 0011, pos = 01, valid = 1
✓ Test Passed: pos = 01, valid = 1
Test 7: in = 0110, pos = 10, valid = 1
✓ Test Passed: pos = 10, valid = 1
Test 8: in = 1100, pos = 11, valid = 1
✓ Test Passed: pos = 11, valid = 1
Test 9: in = 1111, pos = 11, valid = 1
✓ Test Passed: pos = 11, valid = 1
Test 10: in = 1010, pos = 11, valid = 1
✓ Test Passed: pos = 11, valid = 1
Test 11: in = 0101, pos = 10, valid = 1
✓ Test Passed: pos = 10, valid = 1
Test 12: in = 1001, pos = 11, valid = 1
✓ Test Passed: pos = 11, valid = 1
Test 13: in = 0111, pos = 10, valid = 1
✓ Test Passed: pos = 10, valid = 1
Test 14: in = 1110, pos = 11, valid = 1
✓ Test Passed: pos = 11, valid = 1
Test 15: in = 0100, pos = 10, valid = 1
✓ Test Passed: pos = 10, valid = 1
Test 16: in = 1101, pos = 11, valid = 1
✓ Test Passed: pos = 11, valid = 1
Test 17: in = 1011, pos = 11, valid = 1
✓ Test Passed: pos = 11, valid = 1
Test 18: in = 1000, pos = 11, valid = 1
✓ Test Passed: pos = 11, valid = 1
Test 19: in = 0010, pos = 01, valid = 1
✓ Test Passed: pos = 01, valid = 1
Test 20: in = 0001, pos = 00, valid = 1
✓ Test Passed: pos = 00, valid = 1
Test Summary: Total = 20, Passed = 20, Failed = 0

================================================================================
```